CHAPTER 5

Beyond the App: Adding Redis

OK, fess up. Did you look at the title of this chapter and think "Redis?! What about setting up the database?!" If so, I promise I'm not crazy: there's a very good reason for tackling Redis first, as you'll discover shortly.

First, let's review what we've achieved. We've already learned how to:

- Use Docker to generate a fresh Rails project without Ruby installed
- Start the Ruby server to run our application
- Ensure our gems are installed and up to date
- Create our very own Docker image suited to running our Rails app
- Use Docker Compose to manage the whole process

That's not a bad start, but currently it isn't enough to build anything other than the most basic of websites or apps. We're missing a key piece of the puzzle: how to connect our Rails app to external services like…a database. In this chapter, you'll learn how to do just that, starting with Redis (if you're not familiar with Redis, it's an in-memory, key-value store commonly used for pub/sub messaging, queues, caching, and more—all the cool kids use it, and after this chapter, you will too).

Why Redis before a database? Because, while the process of adding services to our app is similar, it turns out that Redis is *easier to integrate* into our app than a database. Take my word for it: doing it this way around will give you a smoother ride.

In fact, this chapter teaches the basic skills to add *any* service to your app, be it a database (which we'll do in the very next chapter), background workers, Elasticsearch, or even a separate JavaScript front end. Soon our Docker-fueled apps will be every bit as powerful as we're used to, and then some.